



كلية الهندسة المعلوماتية

جامعة دمشق

السنة الرابعة (الذكاء الصناعي)

مشروع البرمجة التفرعية

الطلاب:

1. محمد جمال أيمن البلخي
2. شهد عماد بريغش
3. شهد رامح معلا
4. آلاء محمد يسر دلعو
5. محمد يوسف عبدالكريم

مقدمة

يهدف المشروع إلى بناء نظام متكامل لمنظومة تجارة إلكترونية تتعامل مع آلاف الطلبات المتزامنة ومقارنة أداء النظام وتطبيق المفاهيم المتعلقة بالمتطلبات غير الوظيفية مثل الأداء، حماية البيانات من التضارب، إدارة الموارد، وأتمتة العمليات الخلفية.

المتطلبات الوظيفية:

- 1- إنشاء حساب
- 2- تسجيل دخول
- 3- إدارة مخزون
- 4- إضافة للسلة (create order)
- 5- دفع
- 6- توليد تقارير دورية للـ admin

المتطلبات غير الوظيفية حسب ملف المشروع:

- 1- حماية البيانات المشتركة من التضارب:
التضارب في الوصول المتزامن إلى البيانات (Race Condition) أحد أخطر التحديات عند السماح لعدة مستخدمين أو عمليات بخدمة نفس المورد في الوقت نفسه. تظهر هذه المشكلة بوضوح عند تحديث كميات المخزون، حيث قد يقوم مستخدمان بقراءة القيمة نفسها قبل تحديثها، فيظن كل منهما أن الكمية كافية، ثم يقومان بعملية الشراء بالتزامن، مما يؤدي إلى تجاوز الكمية الفعلية المتاحة وفقدان سلامة البيانات. ولا تقتصر هذه المشكلة على المخزون فقط، بل تظهر أيضاً في عمليات الدفع وتحويل الأموال؛ إذ يمكن لعمليتين ماليتين متوازيتين أن تقوموا بقراءة الرصيد نفسه قبل خصمه، مما يؤدي إلى سحب مبلغ أكبر من الرصيد

المتاح أو تنفيذ تحويلين ماليين متداخلين على نفس الحساب، وهو ما يشكل خطراً كبيراً على سلامة النظام المالي.

ولمعالجة هذا النوع من التضارب، يتم اعتماد القفل التشاؤمي (Pessimistic Lock) الذي يمنح العملية الأولى حق الوصول الحصري إلى الصف المطلوب في قاعدة البيانات، ويجبر العمليات الأخرى على الانتظار حتى انتهاء المعاملة. ويتم ذلك باستخدام آلية LockForUpdate التي تقوم بحجز الصف أثناء تنفيذ المعاملة، مما يمنع أي عملية أخرى من قراءته أو تعديله حتى يتم تحرير القفل. وإلى جانب ذلك، يتم تنفيذ العملية كاملة داخل معاملة قاعدة بيانات (DB Transaction) لضمان أن جميع الخطوات المرتبطة بالعملية — مثل خصم الرصيد، تحديث المخزون، أو إنشاء سجل الطلب — تتم كوحدة واحدة متكاملة؛ فإذا فشل أي جزء من العملية يتم التراجع عن جميع التغييرات تلقائياً، مما يمنع حدوث أي نجاح جزئي قد يؤدي إلى بيانات غير متسقة. بهذه الطريقة، يتم ضمان أن عمليات تحديث المخزون أو تحويل الأموال تتم بشكل متسلسل وآمن، وأن النظام يحافظ على تكامل البيانات (Data Integrity) ويمنع تماماً حدوث أي تضارب أو تجاوز في الكميات أو الأرصدة المالية.

يوجد حلول أخرى لكن لم نستخدمها:
استخدام Queue بحيث كل عملية شراء أو دفع تدخل طابور، ويُعالج الطلب تلو الآخر، لكن لم نستخدمها لأن النظام سيتعرض للكثير من الطلبات في وقت معين وهذا يحتاج لسرعة على خلاف ما سيحققه الطابور.
استخدام optimistic lock يسمح للجميع بالقراءة، لكن يمنع الكتابة إذا تغيّر الإصدار لكن لم نستخدمه لأنه فقط سيكشف وجود race condition لكن لن يمنعه لأنه لا يطبق قفل على الـ resource

```

public function utilityTransfer($from, $to, float $amount, $reference, $notes)
{
    return DB::transaction(function () use ($from, $to, $amount, $reference, $notes) {
        // Always lock wallets in the same order to prevent deadlocks
        $walletA = Wallet::where('id', min($from, $to))->lockForUpdate()->firstOrFail();

        $walletB = Wallet::where('id', max($from, $to))->lockForUpdate()->firstOrFail();

        // Now assign them back to from/to
        $fromWallet = $from == $walletA->id ? $walletA : $walletB;
        $toWallet = $to == $walletA->id ? $walletA : $walletB;

        // Check balance AFTER locking
        if ($fromWallet->balance < $amount) { ...
        }

        // Resolve polymorphic types dynamically
        $fromType = get_class($fromWallet->user);
        $toType = get_class($toWallet->user);
        $refType = Order::class;

        $fromName = class_basename($fromType);
        $toName = class_basename($toType);
        $refName = class_basename($reference) ?? null;

        // 1 Debit sender wallet
        WalletTransaction::create([ ...
        ]);

        $fromWallet->decrement('balance', $amount);

        // 2 Credit receiver wallet
        WalletTransaction::create([ ...
        ]);

        $toWallet->increment('balance', $amount);

        // 3 Update order status only if applicable
        if ($reference instanceof Order) {
            $reference->update(['status' => OrderStatusEnum::PAID->value]);
        }
    });
}

```

كما موضح بالكود:

- تم قفل محفظة الادمين والمستخدم بنفس الترتيب الموضح دائماً (الادمين ثم المستخدم) بغض النظر عن الشخص المرسل أو المستقبل لمنع الـ dead lock
- تم إضافة lookForUpdate لمنع الـ race condition

	id	order_id	user_id	product_id	original_price	price	quantity	created_at
e	1	1	3	1	25.00	100.00	4	2026-05-18 18:27:04
e	2	2	4	1	25.00	100.00	4	2026-05-18 18:27:04

id	name	description	price	stock
1	eligendi et ratione	Quia officia voluptas qui dolorum rerum cupiditate...	25.00	4

كما نرى حصل race condition

بعد تطبيق الحلول المذكورة أعلاه، تم حل المشكلة وبنفس الطريقة تم نفس المشكلة في متطلب الدفع.

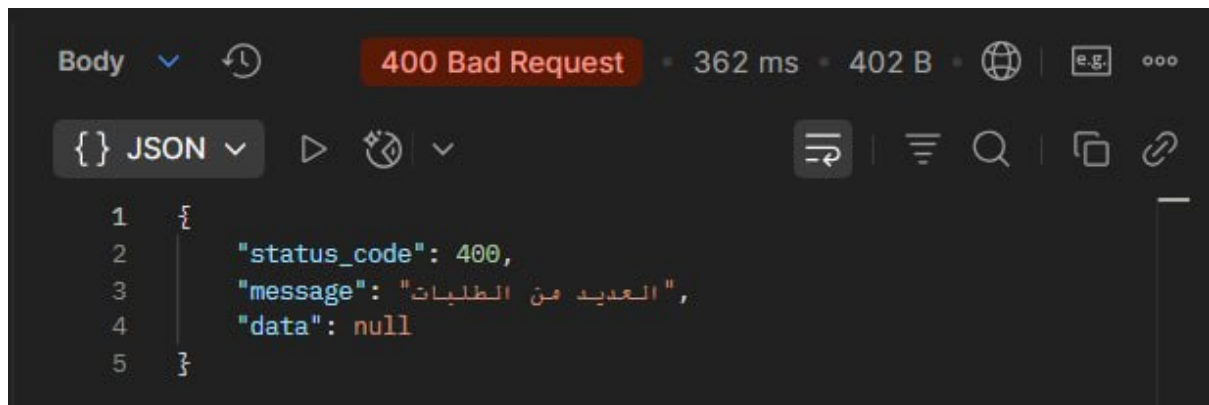
2- إدارة الموارد الحاسوبية:

في الأنظمة عالية الحمل، لا تكمن المشكلة فقط في تنفيذ العمليات المتوازية، بل في التحكم في عددها بحيث لا يستهلك النظام موارده بشكل يؤدي إلى انهيار أو بطء شديد. تظهر هذه المشكلة عندما يصل عدد الطلبات المتزامنة إلى مستوى

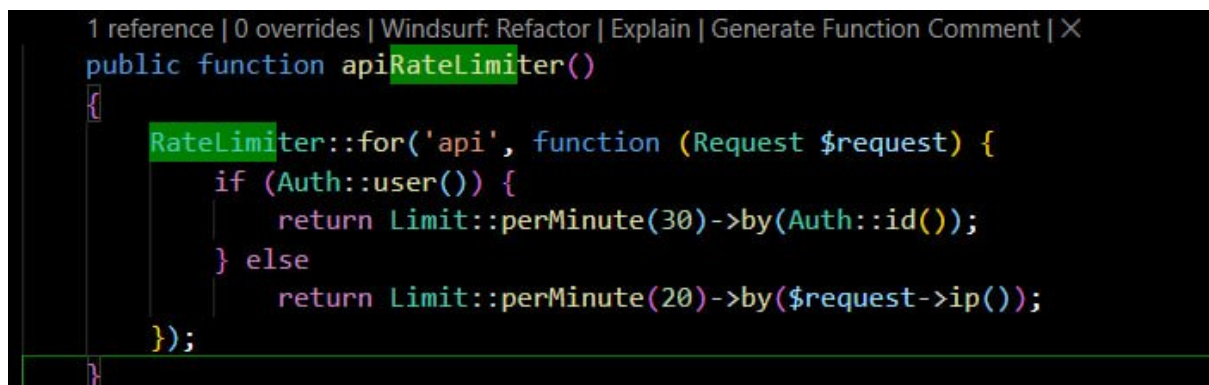
يفوق قدرة الخادم على المعالجة، مما يؤدي إلى استنزاف المعالج والذاكرة وارتفاع زمن الاستجابة. ولحل هذه المشكلة، تم اعتماد مجموعة من الآليات التي تضمن إدارة الموارد الحاسوبية بشكل متوازن ومنع النظام من تجاوز قدرته التشغيلية.

أولاً، تم استخدام محدد المعدل (Rate Limiter) للتحكم في عدد الطلبات التي يمكن للنظام قبولها خلال فترة زمنية محددة، بحيث يتم رفض أو تأجيل الطلبات الزائدة عن الحد المسموح به. هذه الآلية تمنع حدوث طفرات مفاجئة في الحمل وتضمن بقاء النظام ضمن حدود قدرته الفعلية.

كما موضح بالصور:



```
Body 400 Bad Request 362 ms 402 B
JSON
1 {
2   "status_code": 400,
3   "message": "العديد من الطلبات",
4   "data": null
5 }
```



```
1 reference | 0 overrides | Windsurf: Refactor | Explain | Generate Function Comment | X
public function apiRateLimiter()
{
    RateLimiter::for('api', function (Request $request) {
        if (Auth::user()) {
            return Limit::perMinute(30)->by(Auth::id());
        } else {
            return Limit::perMinute(20)->by($request->ip());
        }
    });
}
```

ثانياً، تم استخدام خيط معالجة مخبأ (Cached Thread Pool) لإدارة المهام القصيرة والمتكررة بكفاءة عالية (add to cart + transfer payment)، حيث يقوم هذا النوع بإعادة استخدام الخيوط الجاهزة بدلاً من إنشاء خيوط جديدة في كل مرة، مما يقلل من تكلفة إنشاء الخيوط ويحسن استهلاك الموارد. كما تم استخدام Scheduled Thread Pool لجدولة المهام الدورية مثل معالجة الدُفعات (إصدار تقارير)، مما يسمح بتنفيذ المهام الخلفية دون التأثير على المسار الرئيسي للطلبات.

```
'environments' => [
  'production' => [
    // Here you can define the supervisor settings for production environment

    //cart supervisor , cached thread pool, auto scaling , number of workers range from 4 to 10
    'supervisor-cart' => [
      'connection' => 'redis',
      'queue' => ['cart'],
      'balance' => 'auto',
      'minProcesses' => 4,
      'maxProcesses' => 10,
    ],

    //transfer supervisor , cached thread pool, auto scaling , number of workers range from 4 to 10
    'supervisor-transfer' => [
      'connection' => 'redis',
      'queue' => ['transfer'],
      'balance' => 'auto',
      'minProcesses' => 4,
      'maxProcesses' => 10,
    ],

    //report supervisor , fixed thread pool, 2 workers
    'supervisor-report' => [
      'connection' => 'redis',
      'queue' => ['report'],
      'balance' => 'simple',
      'processes' => 2, // fixed
    ],

    //dead letter supervisor , fixed thread pool, 1 worker
    'supervisor-dead-letter' => [
      'connection' => 'redis',
      'queue' => ['dead-letter'],
      'balance' => 'simple',
      'processes' => 1, // fixed
    ],
  ],
],
```

وأخيراً، تم دمج نمط الدارة المعطلة (Circuit Breaker) لحماية النظام من الانهيار عند تكرار فشل خدمة معينة. فعندما تتجاوز نسبة الفشل الحد المسموح به،

يتم إيقاف إرسال الطلبات إلى تلك الخدمة مؤقتاً، مما يمنع استنزاف الموارد ويحافظ على استقرار بقية أجزاء النظام. وعند انتهاء فترة التبريد، يتم اختبار الخدمة تدريجياً قبل إعادة فتح الدارة بالكامل.

كما في الصورة:


```

class CircuitBreakerMiddleware
{
    /**
     * Handle an incoming request.
     *
     * @param \Closure(\Illuminate\Http\Request): (\Symfony\Component\HttpFoundation\Response) $next
     */
    public function handle($request, Closure $next, $serviceName)
    {
        $key = "circuit_{$serviceName}";

        $state = Cache::get($key, [
            'failures' => 0,
            'last_failure' => null,
            'open' => false,
        ]);

        // Circuit OPEN
        if ($state['open']) {
            // Cooldown passed + HALF-OPEN
            if ($state['last_failure'] && now()->diffInSeconds($state['last_failure']) >= 30)
                return $this->attempt($next, $request, $key, $state);

            // if still in cooldown + return error
            return response()->json([
                'status' => 'blocked',
                'message' => "{$serviceName} temporarily unavailable"
            ], 503);
        }

        // Circuit CLOSED
        return $this->attempt($next, $request, $key, $state);
    }

    protected function attempt($next, $request, $key, $state)
    {
        try {
            $response = $next($request);

            // Success + close circuit
            Cache::put($key, [
                'failures' => 0,
                'last_failure' => null,
                'open' => false,
            ]);

            return $response;
        } catch (\Exception $e) {
            // Failure + update state
            $state['failures']++;
            $state['last_failure'] = now();

            // Open circuit if failures exceed threshold
            if ($state['failures'] >= 3)
                $state['open'] = true;

            // Save state and return error
            Cache::put($key, $state);

            return unavailableServiceFailure([], "Service is currently unavailable. Please try again later.");
        }
    }
}

```

الكود الظاهر في الصورة يمثل آلية (Circuit Breaker) داخل وسيط (Middleware) في Laravel، وهي تقنية تُستخدم لحماية النظام من تكرار الأخطاء عند فشل خدمة خارجية أو داخلية.

يقوم الكود بتخزين حالة الخدمة في الذاكرة المؤقتة (Cache) عبر مفاتيح تحتوي على عدد الإخفاقات، آخر وقت فشل، وحالة القاطع (مفتوح أو مغلق). عند كل

طلب، يتحقق الوسيط مما إذا كانت الدارة مفتوحة — أي أن الخدمة فشلت عدة مرات متتالية — فيمنع الطلبات الجديدة مؤقتاً ويعيد استجابة بخطأ 503. بعد مرور فترة تهدئة (Cooldown)، يسمح بمحاولة واحدة لاختبار الخدمة من جديد. إذا نجحت المحاولة، تُغلق الدارة ويُعاد ضبط العدّاد، أما إذا فشلت، فيُزاد عدد الإخفاقات حتى يصل إلى الحد الأقصى (3 مرات مثلاً)، فيُفتح القاطع مجدداً ويُوقف الطلبات مؤقتاً. بهذه الطريقة، يمنع الكود النظام من استنزاف الموارد أو تكرار الاتصال بخدمات غير مستقرة، مما يحافظ على استقرار الأداء وموثوقية النظام.

من خلال الجمع بين هذه الآليات، يتمكن النظام من التحكم في مستوى التوازي، ومنع الحمل الزائد، وضمان استهلاك متوازن للموارد حتى تحت الضغط العالي، مما يحقق استجابة مستقرة وموثوقة في بيئة تشغيل متزامنة.

3- المعالجة غير المتزامنة:

في هذا المتطلب المتعلق بالمعالجة غير المتزامنة (Asynchronous)

(Queues)، يهدف الحل إلى فصل المهام الثقيلة أو غير الفورية عن المسار الرئيسي لطلب المستخدم، بحيث لا ينتظر المستخدم انتهاء العمليات الخلفية الطويلة مثل إصدار الفواتير، إرسال الإشعارات، أو تحديث التقارير.

تم اعتماد نظام الطوابير (Queue Jobs) لتنفيذ هذه المهام في الخلفية بطريقة منظمة وآمنة، حيث يتم وضع كل مهمة في طابور خاص ليتم تنفيذها بواسطة عامل (Worker) مستقل دون التأثير على سرعة استجابة الطلبات الأساسية. هذه الآلية تسمح للنظام بمعالجة آلاف العمليات بالتوازي دون أن يشعر المستخدم بأي تأخير، كما تضمن توزيع الحمل بشكل متوازن بين الخيوط العاملة.

وبذلك، تتحقق المعالجة غير المتزامنة بشكل فعال، إذ يتم تنفيذ المهام الثقيلة في الخلفية بينما يستمر النظام في خدمة الطلبات الجديدة بسرعة، مما يؤدي إلى تحسين الأداء العام، تقليل زمن الاستجابة، وزيادة استقرار النظام تحت الضغط العالي.

```
references | 0 implementations | Windsurf: Refactor | Explain
class TransferJob implements ShouldQueue

use Dispatchable, InteractsWithQueue, Queueable, SerializesModels;

0 references
public $tries = 5;
0 references
public $backoff = [5, 15, 30, 60, 120];
0 references | 0 overrides | Windsurf: Refactor | Explain | Generate Function Comment | X
public function __construct(
    public int $from,
    public int $to,
    public float $amount,
    public $reference,
    public ?string $notes
) {
    // Set the queue name for this job
    $this->onQueue('transfer');
}

0 references | 0 overrides | Windsurf: Refactor | Explain | Generate Function Comment | X
public function handle()
{
    // Perform the transfer logic
    app(WalletService::class)->utilityTransfer(
        $this->from,
        $this->to,
        $this->amount,
        $this->reference,
        $this->notes
    );
}

0 references | 0 overrides | Windsurf: Refactor | Explain | Generate Function Comment | X
public function failed(\\Throwable $exception)
{
    // Log the failure and dispatch to dead letter queue
    dispatch(new DeadLetterJob([
        'job' => self::class,
        'payload' => [
            'from' => $this->from,
            'to' => $this->to,
            'amount' => $this->amount,
            'reference' => $this->reference,
            'notes' => $this->notes,
        ],
        'error' => $exception->getMessage(),
    ]))->onQueue('dead-letter');
}

31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97

2 references | 0 overrides | Windsurf: Refactor | Explain | Generate Function Comment | X
public function transfer($from, $to, float $amount, $reference, $notes)
{
    TransferJob::dispatch(
        $from,
        $to,
        $amount,
        $reference,
        $notes
    );
}

1 reference | 0 overrides | Windsurf: Refactor | Explain | Generate Function Comment | X
public function utilityTransfer($from, $to, float $amount, $reference, $notes)
{
    return DB::transaction(function () use ($from, $to, $amount, $reference, $notes) {
        // Always lock wallets in the same order to prevent deadlocks
        $walletA = Wallet::where('id', min($from, $to))->lockForUpdate()->first();
        $walletB = Wallet::where('id', max($from, $to))->lockForUpdate()->first();

        // Now assign them back to from/to
        $fromWallet = $from == $walletA->id ? $walletA : $walletB;
        $toWallet = $to == $walletA->id ? $walletA : $walletB;

        // Check balance AFTER locking
        if ($fromWallet->balance < $amount) { ...
        }

        // Resolve polymorphic types dynamically
        $fromType = get_class($fromWallet->user);
        $toType = get_class($toWallet->user);
        $refType = Order::class;

        $fromName = class_basename($fromType);
        $toName = class_basename($toType);
        $refName = class_basename($reference) ?? null;

        // Debit sender wallet
        WalletTransaction::create([ ...
        ]);

        $fromWallet->decrement('balance', $amount);

        // Credit receiver wallet
        WalletTransaction::create([ ...
        ]);

        $toWallet->increment('balance', $amount);

        // Update order status only if applicable
        if ($reference instanceof Order) {
            $reference->update(['status' => OrderStatusEnum::PAID, 'paid_at' => now()]);
        }
    });
}
```

الكود الظاهر يمثل منطق نقل الأموال (Transfer) في نظام إلكتروني يعتمد على Laravel، ويُنفَّذ العملية بشكل آمن ومتزامن عبر الطوابير (Queue Jobs).

الجزء الأيسر يوضح تعريف مهمة الخلفية TransferJob التي تُنفَّذ عملية التحويل في الخلفية باستخدام نظام الطوابير، مع تحديد عدد المحاولات، فترات التراجع (Backoff)، وآلية التعامل مع الفشل عبر إرسال المهمة إلى طابور الرسائل الميتة (Dead Letter Queue) لتسجيل الأخطاء وتحليلها لاحقاً.

أما الجزء الأيمن فيُظهر منطق التحويل داخل WalletService، حيث يتم تنفيذ العملية داخل معاملة قاعدة بيانات (DB Transaction) لضمان أن العملية إما تنجح بالكامل أو تُلغى بالكامل دون أي تحديث جزئي. كما يتم استخدام القفل التشاؤمي (lockForUpdate) لحجز صفوف المحافظ المالية أثناء التحديث، مما يمنع أي عملية أخرى من تعديل الرصيد في نفس الوقت ويمنع حدوث تضارب أو تجاوز في الأرصدة.

بعد التأكد من الرصيد، يتم خصم المبلغ من محفظة المرسل وإضافته إلى محفظة المستلم، مع إنشاء سجلين في جدول المعاملات لتوثيق العملية، ثم تحديث حالة الطلب إلى "مدفوع" إن وُجد مرجع للطلب.

```

1  <?php
2
3  namespace App\Jobs;
4
5  use App\Jobs\DeadLetterJob;
6  use App\Services\Wallet\WalletService;
7  use Illuminate\Contracts\Queue\ShouldQueue;
8  use Illuminate\Foundation\Bus\Dispatchable;
9  use Illuminate\Foundation\Queue\Queueable;
10 use Illuminate\Queue\InteractsWithQueue;
11 use Illuminate\Queue\SerializesModels;
12
13 3 references | 0 implementations | Windsurf: Refactor | Explain
14 class TransferJob implements ShouldQueue
15 {
16     use Dispatchable, InteractsWithQueue, Queueable, SerializesModels;
17
18     0 references
19     public $tries = 5;
20     0 references
21     public $backoff = [5, 15, 30, 60, 120];
22     0 references | 0 overrides | Windsurf: Refactor | Explain | Generate Function Comment | ✕
23     public function __construct(
24         public int $from,
25         public int $to,
26         public float $amount,
27         public string $reference,
28         public ?string $notes
29     ) {
30         // Set the queue name for this job
31         $this->onQueue('transfer');
32     }
33
34     0 references | 0 overrides | Windsurf: Refactor | Explain | Generate Function Comment | ✕
35     public function handle()
36     {
37         // Perform the transfer logic
38         app(WalletService::class)->utilityTransfer(
39             $this->from,
40             $this->to,
41             $this->amount,
42             $this->reference,
43             $this->notes
44         );
45     }
46
47     0 references | 0 overrides | Windsurf: Refactor | Explain | Generate Function Comment | ✕
48     public function failed(\Throwable $exception)
49     {
50         // Log the failure and dispatch to dead letter queue
51         dispatch(new DeadLetterJob([
52             'job' => self::class,
53             'payload' => [
54                 'from' => $this->from,
55                 'to' => $this->to,
56                 'amount' => $this->amount,
57                 'reference' => $this->reference,
58                 'notes' => $this->notes,
59             ],
60             'error' => $exception->getMessage(),
61         ]))->onQueue('dead-letter');
62     }
63 }

```

الكود المعروض يحقق المفاهيم الثلاثة الموضحة في الصورة الثانية بشكل عملي داخل نظام الطوابير في Laravel.

ف عند حدوث فشل في المعالجة (Processing Failure) — مثل انقطاع الاتصال بقاعدة البيانات أو خطأ في التنفيذ — يتم إعادة المهمة إلى الطابور تلقائياً لتُعاد المحاولة باستخدام آلية إعادة المحاولة (Retry Mechanism) مع فترات انتظار متزايدة تدريجياً (Exponential Backoff) كما هو محدد في مصفوفة \$backoff. وإذا فشلت جميع المحاولات بعد العدد الأقصى المحدد في \$tries، يتم إرسال المهمة إلى طابور الرسائل الميتة (Dead Letter Queue) عبر مهمة DeadLetterJob لتسجيل تفاصيل الخطأ وتحليلها لاحقاً.

بهذا الأسلوب، يضمن الكود معالجة المهام بشكل موثوق، مع نظام ذكي لإعادة المحاولة والتعامل مع الفشل دون فقدان البيانات أو توقف النظام.

بهذا الأسلوب، يضمن الكود تنفيذ عمليات التحويل المالي بشكل آمن، متسلسل، ومتزامن دون فقدان بيانات أو تضارب في الأرصدة.

4- معالجة البيانات الضخمة على دفعات:

في هذا المتطلب الخاص بمعالجة البيانات الضخمة على دفعات (Batch Processing)، يهدف الحل إلى تنفيذ المهام الثقيلة مثل توليد تقارير المبيعات اليومية بطريقة منظمة لا تؤثر على أداء النظام أثناء التشغيل. ولتحسين الأداء أكثر، تم استخدام تقنية التقسيم إلى دفعات (Chunk Processing) لتقسيم البيانات الكبيرة إلى أجزاء صغيرة تُعالج تدريجياً، مما يقلل

من استهلاك الذاكرة ويزيد من سرعة التنفيذ.

```
while (true)
{
    $start = microtime(true);

    // Fetch chunk of orders
    $orders = Order::where('id', '>', $lastId)
        ->orderBy('id')
        ->limit($chunk)
        ->get();

    if ($orders->isEmpty()) {
        break;
    }

    // Process each order in the chunk
    foreach ($orders as $order) {
        $orderDetails[] = [
            'order_id' => $order->id,
            'total_items' => $order->items()->sum('quantity'),
            'total_price' => $order->items()->sum('price'),
            'average_item_price' => $order->items()->avg('price'),
            'created_at' => $order->created_at,
        ];
    }

    // Update last processed ID
    $lastId = $orders->last()->id;

    // Measure processing time
    $duration = microtime(true) - $start;

    // Dynamic chunking logic
    if ($duration < 0.5 && $chunk < $maxChunk) {
        $chunk += 100; // speed is good + increase chunk size
    }

    if ($duration > 1.5 && $chunk > $minChunk) {
        $chunk -= 50; // too slow + decrease chunk size
    }
}

// -----
// 3. FINAL REPORT OUTPUT
// -----

return [
    'orders' => $ordersSummary,
    'products' => $productsSummary,
    'order_items' => $orderItemsSummary,
    'order_details' => $orderDetails, // new heavy section
];
```

تم اعتماد آلية تعتمد على جدولة المهام باستخدام Cron Job لتشغيل عملية توليد التقارير بشكل دوري دون تدخل يدوي، مع نقل عملية التنفيذ إلى نظام الطوابير (Queue) بحيث تُنفَّذ في الخلفية بعيداً عن المسار الرئيسي للطلبات.

كما موضح:

```
//Report Generation Command / cron job
Schedule::command('reports:generate')->dailyAt('04:00');

5 use App\Services\OrderService;
6 use Illuminate\Console\Command;
7
8 0 references | 0 implementations | Windsurf: Refactor | Explain
9 class GenerateReportCommand extends Command
10 {
11     /**
12      * The name and signature of the console command.
13      * @var string
14      */
15     0 references
16     protected $signature = 'reports:generate';
17
18     /**
19      * The console command description.
20      * @var string
21      */
22     0 references
23     protected $description = 'Generate reports and dispatch report jobs';
24
25     /**
26      * Execute the console command.
27      */
28     0 references | 0 overrides | Windsurf: Refactor | Explain | X
29     public function handle()
30     {
31         app(OrderService::class)->generate();
32         $this->info('Report generation triggered.');
```

بهذا الأسلوب، يتمكن النظام من معالجة كميات ضخمة من البيانات بشكل متزامن وآمن، مع الحفاظ على استقرار الأداء واستمرارية الخدمة حتى أثناء فترات الضغط العالي.

5- توزيع الأحمال:

في هذا المتطلب المتعلق بتوزيع الأحمال (Load Distribution)، يهدف الحل إلى ضمان قدرة النظام على التعامل مع عدد كبير من الطلبات المتزامنة عبر توزيعها على عدة خوادم بطريقة ذكية ومتوازنة. ولتحقيق ذلك، تم استخدام واجهة فحص الصحة (Health API) التي تُمكن الموزّع من التحقق من حالة كل خادم قبل إرسال الطلبات إليه، بحيث يتم استبعاد أي خادم غير مستجيب أو يعاني من ضغط مرتفع.

كما تم اعتماد أداة HAProxy كموزّع أحمال رئيسي، مع تصميم إستراتيجية توزيع تعتمد على طبيعة الاستخدام الجغرافي للنظام؛ إذ تم افتراض وجود أكثر من 10 خوادم مخصصة للمستخدمين داخل سوريا نظراً لارتفاع حجم الطلبات، بينما تم تخصيص خادم واحد لكل دولة أخرى. يقوم HAProxy بتوجيه الطلبات القادمة من سوريا إلى مجموعة الخوادم الكبيرة باستخدام خوارزمية مناسبة مثل Least Connections، بينما يتم توجيه الطلبات من بقية الدول إلى خوادمها الخاصة باستخدام خوارزمية مثل IP Hash لضمان ثبات الجلسة.

بهذا الأسلوب، يتم تحقيق توزيع فعال للأحمال، وضمان استمرارية الخدمة، وتقليل زمن الاستجابة، مع الحفاظ على استقرار النظام حتى عند ارتفاع عدد المستخدمين المتزامنين.

كما موضح بالصورة:

```

1 global
2     log /dev/log local0
3     log /dev/log local1 notice
4     chroot /var/lib/haproxy
5     stats socket /run/haproxy/admin.sock mode 660 level admin
6     stats timeout 30s
7     user haproxy
8     group haproxy
9     daemon
10
11     # Load GeoIP Lua script
12     lua-load /etc/haproxy/geoip.lua
13
14 defaults
15     log global
16     mode http
17     option httplog
18     option dontlognull
19     timeout connect 5000
20     timeout client 50000
21     timeout server 50000
22
23 # -----
24 # FRONTEND: COUNTRY-BASED ROUTING
25 # -----
26 frontend http_front
27     bind *:80
28
29     # Detect country using GeoIP
30     http-request set-var(txn.country) lua.geoip_country
31
32     # ACL: If country == SY (Syria)
33     acl from_syria var(txn.country) -m str SY
34
35     # Route based on country
36     use_backend leastconn_cluster if from_syria
37     use_backend iphash_cluster if !from_syria
38
39     default_backend leastconn_cluster
40
41 # -----
42 # BACKEND 1: LEAST CONNECTIONS (SYRIA ONLY)
43 # -----
44 backend leastconn_cluster
45     balance leastconn
46
47     option httpchk GET /health
48     http-check expect status 200
49
50     server web-app-01 10.0.0.11:80 check inter 3000 fall 3 rise 2
51     server web-app-02 10.0.0.12:80 check inter 3000 fall 3 rise 2
52     server web-app-03 10.0.0.13:80 check inter 3000 fall 3 rise 2
53
54 # -----
55 # BACKEND 2: IP-HASH (SOURCE HASHING) FOR GLOBAL
56 # -----
57 backend iphash_cluster
58     balance source # Equivalent to IP-Hash
59
60     option httpchk GET /health
61     http-check expect status 200
62
63     server web-app-01 10.0.0.11:80 check inter 3000 fall 3 rise 2
64     server web-app-02 10.0.0.12:80 check inter 3000 fall 3 rise 2
65     server web-app-03 10.0.0.13:80 check inter 3000 fall 3 rise 2
66
67 #for this code we suppose that in syria we have +10 servers, and in the other countries we have one server in each country, this code is for HAProxy Tool
68

```

الكود المعروض هو إعداد كامل لأداة HAProxy المستخدمة لتوزيع الأحمال بين الخوادم بناءً على موقع المستخدم الجغرافي.

يبدأ الإعداد بتهيئة الإعدادات العامة للنظام، ثم تحميل سكربت GeoIP لتحديد الدولة من عنوان الـ IP الخاص بالمستخدم. في قسم الـ Frontend، يتم تحليل الدولة وإرسال الطلبات إلى الخوادم المناسبة: فإذا كان المستخدم من سوريا، يتم توجيه الطلبات إلى مجموعة خوادم تستخدم خوارزمية Least Connections لتوزيع الحمل على أكثر من عشرة خوادم سورية، أما المستخدمون من باقي الدول

فيتم توجيههم إلى خوادمهم الخاصة باستخدام خوارزمية IP-Hash التي تضمن ثبات الجلسة لكل مستخدم.

كما يحتوي الكود على فحص صحي دوري (Health Check) لكل خادم لضمان أن الطلبات تُرسل فقط إلى الخوادم السليمة. بهذه الطريقة، يحقق النظام توزيعاً ذكياً للأحمال، ويضمن استقرار الخدمة وسرعة الاستجابة حتى عند ارتفاع عدد المستخدمين المتزامنين.

:(Aspect-Oriented Programming – AOP)

من خلال إنشاء (Middleware) مخصّص لمراقبة الأداء في النظام. يقوم هذا الوسيط بتتبع كل طلب يدخل إلى التطبيق، ويحسب زمن التنفيذ الكلي، وعدد الاستعلامات إلى قاعدة البيانات، وزمنها الإجمالي، إضافة إلى استهلاك الذاكرة أثناء المعالجة. بعد انتهاء الطلب، يتم إرسال هذه البيانات إلى خدمة PerformanceMonitoringService لتسجيلها وتحليلها لاحقاً. تم تطبيق البرمجة الموجهة بالجوانب (AOP) في النقاط الأكثر تأثيراً على أداء النظام، خصوصاً في واجهات الـ APIs التي تتعامل مع قواعد البيانات أو تتطلب عمليات حسابية معقدة.

يُستخدم هذا المفهوم في قياس أداء عمليات مثل جلب المنتجات، إضافة الطلبات، تعديل أو حذف الطلبات، ومعالجة معاملات المحافظ المالية، لأنها تتضمن استعلامات متعددة وتؤثر مباشرة على سرعة النظام واستقراره. كما يُطبّق في عمليات الإدارة مثل تعديل الأسعار أو تحديث حالة الطلب، وأيضاً في المهام الخلفية مثل المعالجة على دفعات (batch processing).

من خلال هذه الآلية، يتم تسجيل زمن التنفيذ، وعدد الاستعلامات، واستهلاك الذاكرة لكل عملية، مما يساعد على تحليل الاختناقات وتحسين الأداء العام للنظام بشكل دقيق ومنهجي.

مثال:

```
local.INFO: API Performance Monitoring [09:57:13 18-05-2026]
{"aspect":"admin.products.index","route":"user.products.index","method":
:"GET","uri":"api/user/products/get","status_code":200,"duration_ms":6.
28,"memory_peak_mb":2,"db_queries_count":1,"db_queries_time_ms":0.86,"u
ser_id":3,"ip":"127.0.0.1","is_slow":false,"created_at":"2026-05-18
{"09:57:13
```

بهذا الشكل، يتم فصل منطق مراقبة الأداء عن منطق العمل الأساسي للتطبيق، مما يحقق مبدأ الفصل بين الاهتمامات (Separation of Concerns) الذي يُعد جوهر هندسة الـ AOP، ويسمح بإضافة مراقبة الأداء لجميع أجزاء النظام دون تعديل الكود الرئيسي.